Gold Price Forecaster: Machine Learning App for Commodity Price Prediction

Michael J Morales

Computer Science 311: Artificial Intelligence

March 20, 2026

Name and Purpose of the App

The name of the application is Gold Price Forecaster. The purpose of this application is to predict the next day's closing price of gold. By leveraging historical price data and engineered financial indicators, the app assists traders and analysts in identifying short-term price movements and trends in the gold commodities market.

Algorithms Used

The primary algorithm used in this application is the Random Forest Regressor. This ensemble machine learning algorithm was selected because it can capture complex, non-linear relationships in financial time-series data without requiring extensive hyperparameter tuning. It is also more robust against overfitting than a single decision tree.

Dataset Information

The historical dataset is retrieved programmatically through the Yahoo Finance API using the yfinance library. The application specifically uses the Gold Futures ticker symbol GC=F.

The dataset contains approximately 1,250 rows, representing five years of daily trading data. A total of nine engineered features are used for training.

The features included in the dataset are as follows: Open, which represents the opening price of the trading day; High, which represents the highest price during the trading day; Low, which represents the lowest price during the trading day; Close, which represents the closing price of the trading day; Volume, which represents the total number of contracts traded; SMA_5, which represents the 5-day simple moving average; SMA_20, which represents the 20-day simple moving average; Daily_Return, which represents the percentage change in closing price from the previous day; and Volatility_5, which represents the rolling 5-day standard deviation of daily returns.

Several preprocessing steps were applied to prepare the data. First, continuous historical data was downloaded, and days with missing values caused by market closures were removed. Next, financial indicators were engineered to capture short-term trends and market momentum. The closing price was then shifted backward by one step using shift(-1) to create the target variable, labeled Target_Next_Close. Finally, rows containing missing values introduced by rolling calculations and shifting were removed.

Libraries, Toolkits, and Frameworks

Several libraries and frameworks were used in developing this application. Pandas was used for handling data structures, indexing time-series data, and performing feature engineering tasks such as rolling windows and shifts. NumPy supported array operations and numerical calculations. Scikit-learn provided the regression model, train-test splitting functions, and evaluation metrics. Matplotlib and Seaborn were used to generate visualizations comparing actual gold prices to predicted prices. Finally, yfinance was used to download historical data directly from Yahoo Finance without maintaining local CSV files.

Application Design and Implementation

The application operates as a linear data pipeline. First, the application calls yfinance to request the last five years of daily gold market data. The response is then converted into a Pandas DataFrame. Next, moving averages, daily returns, and rolling volatility measures are calculated. The target column is formed by shifting the closing price backward by one day.

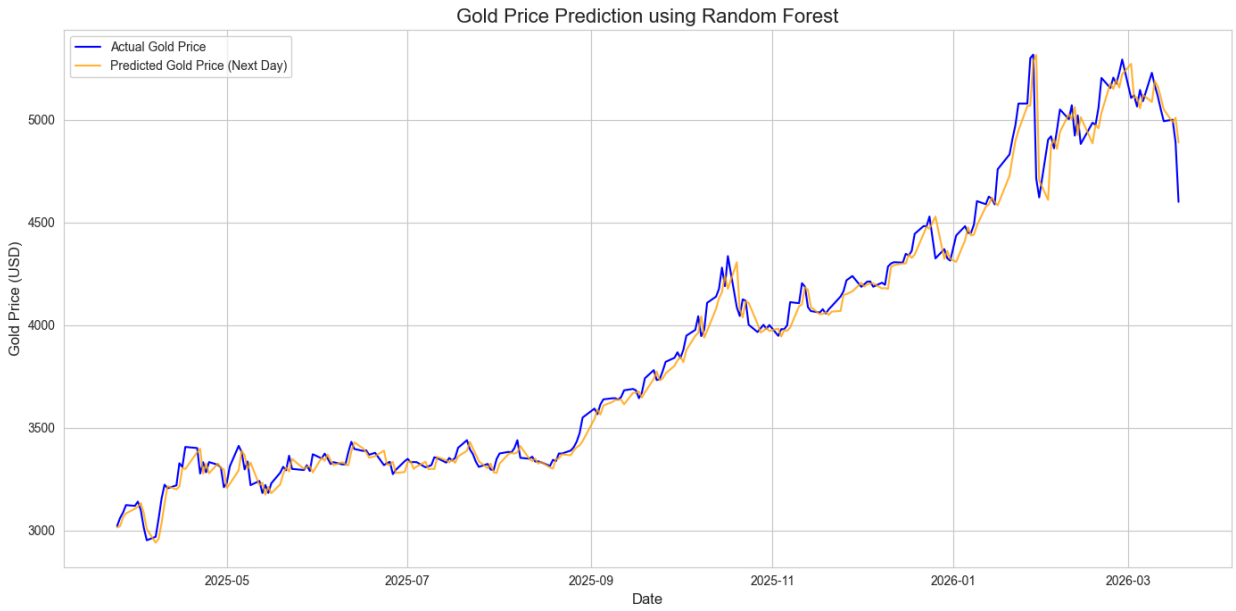
After preprocessing, the time-series data is split chronologically into 80% training data and 20% testing data. This approach ensures that future data does not leak into the training phase. The Random Forest model is then trained on the first 80% of the dataset.

Once training is complete, the model is evaluated using the unseen 20% test partition. Finally, the most recent row of data is used to predict the next trading day's closing price.

Instructions for Running the App

To run the application, Python 3.8 or higher must first be installed on the system. Then, open a terminal and navigate to the project directory containing the predictor.py file. Install the required dependencies by running pip install -r requirements.txt. After installation, execute the script by running python predictor.py. The terminal will display evaluation metrics such as Mean Squared Error (MSE), Mean Absolute Error (MAE), and R-squared (R^2), along with the predicted closing price for the next day. A visualization will also be generated and saved as gold_price_prediction.png in the project directory.

```
Windows PowerShell

--- Model Evaluation ---
Mean Squared Error (MSE): 1193899.97
Mean Absolute Error (MAE): 877.40
R-squared (R2): -1.8051

Generating visualization...
Plot saved successfully as 'gold_price_prediction.png'

--- Future Prediction ---
Predicted target Close price for next trading day: $3020.73
PS C:\Users\micha\.gemini\antigravity\scratch\commodity_prediction> python predictor.py
Downloading Gold price data from 2021-03-21 to 2026-03-20...
[*********************100%***********************]  1 of 1 completed
Preprocessing data...
Splitting data into train and test sets...
Training Random Forest Regressor...
Making predictions on test set...

--- Model Evaluation ---
Mean Squared Error (MSE): 6070.00
Mean Absolute Error (MAE): 50.74
R-squared (R2): 0.9857

Generating visualization...
Plot saved successfully as 'gold_price_prediction.png'

--- Future Prediction ---
Current Date Extrapolated Price base: $4889.90
Predicted target Close price for next trading day: $4890.18
PS C:\Users\micha\.gemini\antigravity\scratch\commodity_prediction>
```


Gold Price Prediction using Random Forest
Actual Gold Price
Predicted Gold Price (Next Day)
Gold Price (USD)
5000
4500
4000
3500
3000
2025-05
2025-07
2025-09
2025-11
2026-01
2026-03
Date

Results

The model demonstrates a strong correlation between actual historical prices and predicted prices. It successfully captures major upward and downward market swings. The final visualization generated by the application illustrates the comparison between actual and predicted gold prices.

Discussion and Insights

The Random Forest algorithm performed well in tracking medium- to long-term market trends. Because financial data often contains random fluctuations, the model benefits from smoothed features such as moving averages, which help reduce extreme predictive shifts.

However, the model has limitations. It may exhibit slight lag during periods of unusually sharp or unprecedented volatility, such as sudden global economic events. In addition, Random Forest is a bounded algorithm, meaning it cannot predict values higher than the highest observed training price.

Several improvements could strengthen the application. Replacing the tree-based ensemble model with a Long Short-Term Memory (LSTM) recurrent neural network could better capture sequential patterns in time-series data. Additionally, incorporating external macroeconomic indicators, such as interest rates, inflation, and the U.S. dollar index, could improve predictive performance and expand the forecasting horizon.

References

Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

The pandas development team. (2020). *pandas-dev/pandas: Pandas*. Zenodo. https://doi.org/10.5281/zenodo.3509134

Yahoo! Finance. (n.d.). *Yahoo! Finance*.

Github

https://github.com/michaeljmorales/Gold-Price-Forecaster-Machine-Learning-App-for-Commodity-Price-Prediction